UNIVERSITY GRADUATE SPECIFICATION TECHNICAL ARTIFACT

ONLINE VOTING SYSTEM

COMBINED SRS & SDD

Architecture Focus: MERN Stack (MongoDB, Express.js, React, Node.js) **Standard Guidelines:** IEEE Std 830-1998 / ISO/IEC/IEEE 29148

Version: 4.0.0 (Production-Ready Complete Edition)

MODULE INDEX & NAVIGATION MATRIX

This comprehensive document integrates the Software Requirements Specification (SRS) and Software Design Document (SDD) across 35 technical modules to provide an ultra-level, clear blueprint for development and deployment.

Part 1: Software Requirements Specification (SRS)

- Modules 3–5: Introduction, Project Boundaries, Definitions, and Architectural Scope
- Modules 6–8: Overall Description, Actor Profiles (Voter, Admin, Officer), and Use Case Layouts
- Modules 9–13: Comprehensive Functional Requirements & Deep-Dive System Use Cases
- Modules 14–16: User Interfaces, Hardware Constraints, and Ecosystem Requirements
- Modules 17–18: Extended Object Classes and Rigorous Non-Functional Performance Vectors

Part 2: Software Design Document (SDD)

- Modules 19–21: Design System Intro, Component Mapping, and Tiered Architecture Layers
- Modules 22–24: System Architecture & Structural Breakdown (Client-to-Database Mapping)
- Modules 25–28: Complete Database Schema Design (MongoDB Deep Collection Layouts)
- Modules 29–31: User Interface Layout Layouts, Frontend Routing, and Navigation Matrix
- Modules 32–34: Transactional Sequences, Verification Plans, and Strategic Compliance
- Module 35: System Evaluation Matrix and Appendix References

1. SRS INTRODUCTION & SYSTEM OBJECTIVES

1.1 Project Purpose

The core objective of this project is to develop a decentralized, secure, transparent, and highly scalable **MERN Stack Online Voting System**. Traditional voting ecosystems suffer from systemic vulnerabilities including physical access constraints, lack of verifiable audit trails, lengthy processing windows, and high operational overhead. This software establishes an online election infrastructure leveraging real-time data synchronization, cryptographic hashing, and automated tally routines.

1.2 Document Scope

This document acts as the definitive source of truth for engineering teams, quality assurance personnel, and university evaluators. It defines the complete operational parameters, user interaction thresholds, interface layouts, relational schemas, and behavioral workflows required to transition the application from a staging architecture to a production environment.

> Target Framework Stack: MongoDB v6.0+, Express.js v4.18+, React v18.2+, Node.js v18.x LTS. Security parameters conform to OWASP Top 10 standards.

2. SYSTEM BOUNDARIES & CONSTRAINTS

Defining clear system boundaries ensures that the scope remains manageable while guaranteeing integration safety with external identity verification systems.

2.1 Core System Boundaries

The system encompasses all workflows executed within the client dashboard (React) and the backend REST API engine (Node.js/Express). It provides dedicated routes for user profile registration, biometric token validation simulations, secure cryptographic ballot casting, live tracking dashboards, and authorized administrative reporting pipelines.

2.2 External Systems Integration

The boundaries terminate at the interface nodes for external verification, payment gateways for registration fees (if applicable), and SMS/Email SMTP gateways for multi-factor authorization. These systems are handled via robust asynchronous webhook integrations.

In-Scope Domains	Out-of-Scope Domains	Integration Strategy
Ballot Hashing, Verification, Dashboard UI, Tokenization	National ID Registry Database hosting, physical identity checking	RESTful JSON Payload exchanges over HTTPS TLS 1.3
Admin Configuration, Real-time Tallying, Fraud Analytics	Hardware terminal engineering, specialized voting machines	Device-agnostic progressive web app distribution

3. GLOSSARY, TECHNICAL ACRONYMS & REFERENCES

3.1 Definitions & Acronyms

Term / Acronym	Comprehensive Technical Definition
MERN	A modern development stack comprising MongoDB (NoSQL Database), Express.js (Backend Web Application Framework), React (Frontend UI Library), and Node.js (JavaScript Runtime Environment).
JWT	JSON Web Token. An open standard (RFC 7519) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object. Used for stateless session tracking.
RBAC	Role-Based Access Control. A method of restricting system access to authorized users based on specific assigned roles (e.g., Voter, Election Officer, Administrator).
Bcrypt	A password-hashing function designed by Niels Provos and David Mazières, based on the Blowfish cipher, incorporating a salt to protect against rainbow table exploits.
SRS / SDD	Software Requirements Specification and Software Design Document, respectively. Standardized IEEE documentation frameworks.

3.2 Reference Standards

IEEE Standard 830-1998 Recommended Practice for Software Requirements Specifications. ISO/IEC 27001 Information Security Management Systems standards for cryptographic safety protocols and data masking models.

4. OVERALL DESCRIPTION & STRATEGIC ADVANTAGES

4.1 General System Workflow Overview

The application runs as an ultra-responsive single-page application (SPA) optimized for mobile, desktop, and tablet displays. Voters interact with an abstract, highly visual user interface that strips out complex background operations. Behind the scenes, every vote is encapsulated as a non-modifiable database transaction signed with a unique session identifier hash, preventing multi-casting vectors and ensuring audit integrity.

4.2 Critical Strategic Advantages

- **Immutable Integrity:** Once a ballot is submitted to the Express.js validation layer, it undergoes validation against a database constraint sequence, rendering double-voting impossible.
- **Zero-Trust Architecture:** Every route request requires token verification, and administrative actions are logged to dedicated system-level monitoring collections.
- **High Availability & Cost Efficiency:** Built entirely on asynchronous JavaScript pipelines, allowing thousands of simultaneous voters to access the system without blocking hardware execution loops.

5. TARGETED ACTOR PROFILES & ACCESS RIGHTS

The system separates functional capability through strict Role-Based Access Control (RBAC). Three primary human entities interact with the system.

5.1 User Group Categorization

User Class / Actor	Functional Responsibility within the Ecosystem	Access Boundaries
System Administrator	Manages system states, provisions Election Officers, reviews system audit logs, configurations, and resolves underlying database conflicts.	Global application view and system write access.
Election Officer	Creates specific election instances, structures candidates, inputs candidate manifestos, establishes open/close timestamps, and oversees tallies.	Write/Edit permissions restricted to elections assigned to them.
Voter	Authenticates identity, views active elections, reviews candidate profiles, casts a single ballot per election, and monitors live result tallies.	Read-only for elections, single write execution for individual ballot casting.

5.2 Environment & Operational Assumptions

Users are assumed to have access to a modern web browser supporting ECMAScript 6+ and a stable internet connection. The hosting server is assumed to maintain a persistent connection to the MongoDB Atlas cluster.

6. HIGH-LEVEL CORE SYSTEM FEATURES MATRIX

The system functional profile is segmented into core feature modules that trace directly to backend components. Below is the master requirements matrix:

Module ID	Feature Module Name	Technical Core Description	Execution Priority
FEAT-001	Biometric/MFA Auth	Enforces dual-layered identity assurance via secure JWT payloads combined with secondary verification.	Critical (P0)
FEAT-002	Dynamic Ballot Builder	Allows Election Officers to declare positions, attach markdown bios, and insert candidate profile imagery dynamically.	High (P1)
FEAT-003	Secure Ballot Casting	Performs atomic write transactions to record the vote while marking the individual voter account status as `voted: true`.	Critical (P0)
FEAT-004	Live Tally Engine	Utilizes MongoDB Change Streams or optimized aggregation groupings to stream vote tracking data to dashboards via secure polling or sockets.	Medium (P2)
FEAT-005	Audit Trail Generation	Maintains immutable, un-editable logs tracking exactly when ballots were added without linking to individual voter names.	High (P1)

7. FUNCTIONAL REQUIREMENTS: SYSTEM USE CASES

The operational interactions within the system are captured below via a comprehensive use case blueprint outlining core relationships.

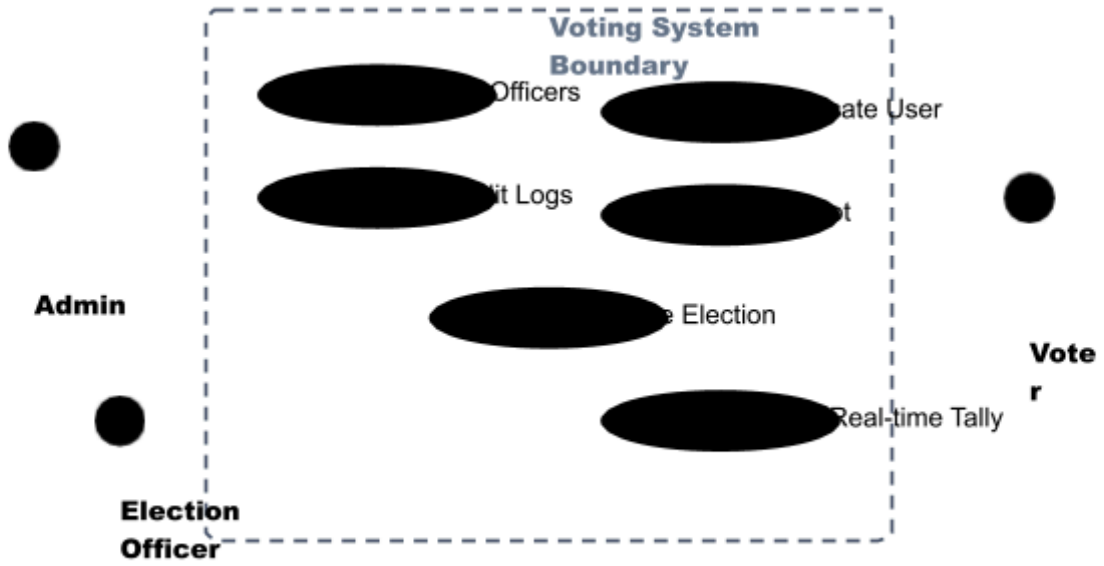


FIGURE 1: HIGH-LEVEL UNIFIED USE CASE RELATIONSHIPS MATRIX

8. USE CASE DEEP DIVE: IDENTITY AUTHENTICATION

This module provides the detailed structural logic governing how users access system functionalities safely.

8.1 Use Case Specifications Table

Field	Detailed Execution Parameter Description
Use Case ID / Name	UC-AUTH-01: Authenticate User Session
Primary Actor	Voter, Election Officer, System Administrator
Pre-conditions	The actor must have a structured account recorded in the MongoDB user collection.
Main Flow of Events	1. User inputs verification identifier and password credentials into the React UI. 2. React Client sends a POST payload to `/api/auth/login`. 3. Node controller calls `bcrypt.compare()` against the hashed password stored in MongoDB. 4. Upon matching, a signed JWT payload with role data is returned. 5. React stores token in local memory context and sets security interceptors.
Alternative Flows	If password validation fails, system responds with a HTTP 401 Unauthorized payload, clearing state.
Post-conditions	Actor receives an authorized navigation routing schema matched to their role profile context.

9. USE CASE DEEP DIVE: BALLOT CASTING PROCESS

The critical functional processing module for recording democratic selections cleanly without structural side effects.

Field	Detailed Execution Parameter Description
Use Case ID / Name	UC-VOTE-02: Secure Cryptographic Ballot Casting
Primary Actor	Verified Voter Account
Pre-conditions	The voter must be logged in, their specific profile `isVerified` flag must equal true, and their `hasVoted` value for the current target Election ID must be false.
Main Flow of Events	1. Voter opens active Election Dashboard and reviews candidate list. 2. Voter selects a candidate checkbox and presses "Submit Formal Ballot". 3. Confirmation Modal opens; Voter provides double-confirmation authorization. 4. Post request containing `{ electionId, candidateId }` sent over TLS 1.3. 5. Controller triggers atomic validation matrix checks. 6. Vote object created, detached from Voter identity, and saved. Voter record updated.
Exception Handling	If the database detects `hasVoted: true` already present during the transaction execution, the entire request throws a 409 Conflict error, and the transaction is aborted.
Post-conditions	The voter profile is locked out from re-entering the target ballot pipeline, and the candidate selection tally increments by 1.

10. DATA FLOW ARCHITECTURE: DFD LEVEL 0

The system context view maps total interaction boundaries between external entities and the centralized architectural engine processing boundary.

FIGURE 2: DFD LEVEL 0 CONTEXT BOUNDARY MAP

The external boundaries establish that state transitions can only be triggered via authorized entry parameters. No processing subsystem allows un-authenticated bypass pipelines.

11. DATA FLOW ARCHITECTURE: DFD LEVEL 1

The DFD Level 1 decomposes the system into discrete, non-overlapping server execution processes handling separate transactional domains.

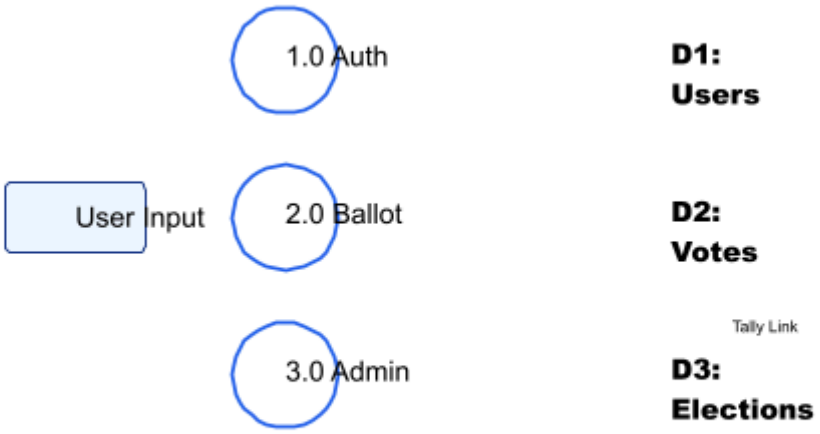


FIGURE 3: DFD LEVEL 1 DETAILED COMPONENT DATA ROUTING SUBSYSTEMS

12. DATA FLOW ARCHITECTURE: DFD LEVEL 2

Isolating the critical **Process 2.0 (Ballot Processing Framework)** down to sub-atomic logic workflows to assure flawless implementation accuracy during programming phases.

FIGURE 4: DFD LEVEL 2 GRANULAR DECOMPOSITION OF VOTE VERIFICATION PIPELINE

This deep decomposition guarantees that checks occur step-by-step. A user token must pass checking before deduplication functions query the active database collections.

13. SYSTEM HARDWARE ENVIRONMENT SPECIFICATIONS

13.1 Production Hosting Infrastructure Matrix

To execute the operational flows without performance degradation, the hardware profiles specified below are required for deployment phases:

Infrastructure Layer	Minimum Target Configuration	Recommended Production Setting
Application Node Hosting	AWS t3.medium Instance (2 vCPUs, 4 GiB Memory Profile)	AWS m6g.large Dedicated Cluster (2 vCPUs, 8 GiB RAM Compute)
Database Core Storage	MongoDB Atlas M10 Shared Cluster Tier	MongoDB Atlas M30 Dedicated Cluster (High IOPS Storage)
Network Pipe Capacity	100 Mbps Dedicated Symmetric Uplink Line	1 Gbps Adaptive Burstable Network Pipeline Interface
End-User Client Footprint	Chrome v90+, Safari v14+, 1GB free client execution RAM	Any HTML5 Compliant Web Runtime with Session Cookie access

13.2 Resource Balancing Strategy

The runtime leverages Node cluster configurations to spin up worker threads scaling exactly to total CPU counts, providing robust horizontal performance metrics.

14. SYSTEM USER INTERFACE & EXPERIENCE DESIGN BLUEPRINT

14.1 UX Strategy Goals

The system styling uses clean layouts focused on structural readability. It abstracts layout variables by ensuring a strict, linear progression across forms. Interface paradigms include high-visibility action states, immediate input error feedback, and robust modal notifications before running final database operations.

14.2 Master Layout Structural Mapping

Dashboard Workspace	Core Visible Grid Modules	Dynamic Route Address
Central Gateway	Header Banner, Identification Input Form, Password Shield, Action Submit Button, Recovery Anchors.	`/login`
Voter Command Hub	Voter Identity Card, Available Contests Loop, Live Summary Sidebar, Historic Vote Verification Tickers.	`/dashboard/voter`
Admin Command Console	Ecosystem Metric Bar charts, Active Officer Matrices, Election Creation Wizards, Audit Log Exporters.	`/dashboard/admin`

> Design Token Palette: Slate Grey Primary (`#1e293b`), Digital Blue Accent (`#2563eb`), Soft Page Canvas Base (`#f8fafc`). Responsive layout break-point set at 768px.

15. SOFTWARE DEVELOPMENT ECOSYSTEM SETTINGS

Ensuring absolute setup parity between collaborative development environments prevents structural runtime divergence during staging phases.

15.1 Component Version Configuration

- **Development Environment Operating Platform:** Ubuntu 22.04 LTS Desktop / Windows 11 Professional Runtime.
- **Runtime Execution Engine Layer:** Node.js Platform Runtime Environment Engine v18.16.0 LTS Edition.
- **Package Management Engine:** npm Ecosystem Dependency Manager v9.5.1+.
- **API Tooling Suite:** Postman Desktop API Testing Environment v10.12+.
- **Local Mock Data Storage Architecture:** MongoDB Community Server Edition v6.0.5 local loop.

15.2 Environment Variable Mapping Specification

```
NODE_ENV=production
PORT=5000
MONGODB_URI=mongodb+srv://cluster0.example.mongodb.net/voting_prod?retryWrites=true&w=majority
JWT_SECRET=8f39b1a0c7e2d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9
TOKEN_EXPIRY_PERIOD=3600s
CORS_ORIGIN_WHITELIST=https://voting-system.university.edu
```

16. SYSTEM NON-FUNCTIONAL REQUIREMENTS (NFR)

The system operation parameters are measured against absolute reliability engineering benchmarks defined below.

16.1 System Security Vectors

All operational database requests are executed behind SSL/TLS 1.3 tunnels. Passwords undergo dynamic salting using a workload factor metric of 2^{12} hashing loops. JSON Web Tokens are short-lived, with expiration values hard-set at $T_{exp} = 3600$ seconds.

16.2 Scalability & Performance Latency Metrics

The API routing controllers are designed to answer active read queries within a strict parameter limit: $T_{latency} \le 200 ext{ms}$ under a base load constraint of 1,500 concurrent server actions.

16.3 High Availability Profile

The target availability index is defined by the formula:

$$A = rac{T_{uptime}}{T_{total}} imes 100 \ge 99.95\%$$

Database backup processes execute via chronological crontab mechanisms every 6 hours, storing snapshots across multi-region object storage endpoints.

17. OBJECT-ORIENTED DOMAIN ARCHITECTURE: CLASS DIAGRAM

The static entity structures mapping attributes, functional execution signatures, and domain associations within the runtime stack are defined below.

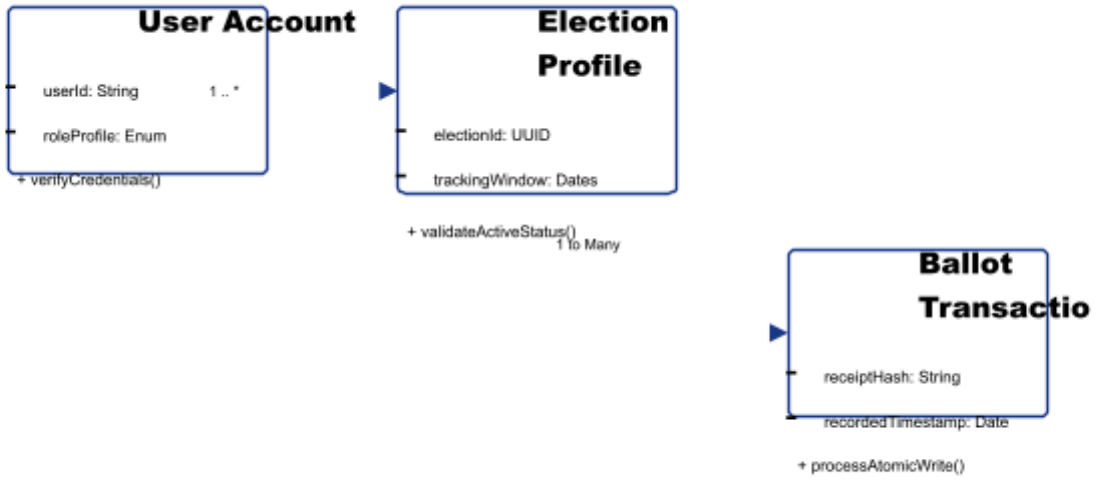


FIGURE 5: UNIFIED DOMAIN MODELING CLASSES AND FUNCTIONAL INTERFACE SIGNATURES

2.0 SDD ARCHITECTURE SYSTEMS SUMMARY

20.1 Structural Integration Purpose

This section transitions the specifications into physical development code guidelines. The Software Design Document (SDD) maps data models, route structures, controller logic blocks, and presentation components to fulfill the requirements detailed in Part 1.

20.2 Overall Structural Pattern

The application implements a decoupled, modern multi-tier software architecture. The frontend tier remains stateless, relying on local contexts for authorization management, while the backend relies on an enterprise-grade Model-View-Controller (MVC) blueprint adapted for RESTful JSON serialization layers.

> Database collections feature unique field structural constraints, guaranteeing document indexing safety directly within the MongoDB storage layer engine.

21. COMPONENT DECOMPOSITION LAYOUT MATRIX

The operational software layers are divided into independent modules that process parameters cleanly through standardized communication paths.

Component Code	Component Domain	Functional Processing Mission	Dependency Layer
COMP-CLI-01	React UI Presentation	Renders virtual layout trees, manages dashboard layout matrices, tracks input states.	Axios Engine, Tailwind
COMP-API-02	Express Routing Controllers	Parses URLs, checks security scopes, manages middleware validations.	Node Core Server
COMP-MID-03	Token Guard Middleware	Inspects request headers, decrypts JWT payloads, throws errors for expired claims.	JsonWebToken Engine
COMP-DAT-04	Mongoose Data Adapters	Enforces structural type validation schemas, maps queries to BSON records.	MongoDB Cluster Cluster

22. CORE SYSTEM ARCHITECTURE DIAGRAM

The physical breakdown of communication tracking paths from the client device through logical gateways down to the data storage volume layer.

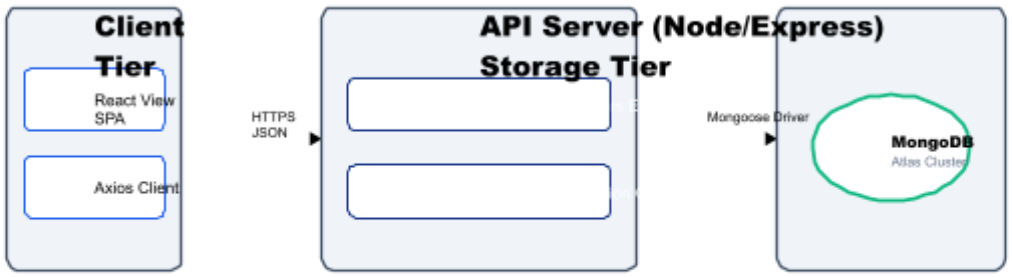


FIGURE 6: TIERED ARCHITECTURAL COMMUNICATION NETWORK FRAMEWORK

23. DEEP ARCHITECTURAL BREAKDOWN & REQUEST LIFE CYCLE

To prevent structural breakdowns during traffic spikes, requests pass through a sequential validation funnel:

23.1 Detailed Step-by-Step Processing Lifecycle

1. **Ingress Phase:** The user request strikes the Express listener module port. The engine extracts the dynamic path variables and payload body parameters.
2. **Authentication Pre-processing:** The system fires the JWT header interceptor card. If no bearer token is parsed, processing stops immediately, throwing an un-authenticated exception.
3. **Validation Filtering:** The payload drops into the schema checker block. Input values are sanitized to protect against cross-site scripting (XSS) or injection attacks.
4. **Data Transaction Mapping:** The clean request context is passed to Mongoose database adaptors, which execute optimized indexing lookups against the MongoDB cluster storage engines.

> Every response lifecycle step guarantees that resources clear memory space as soon as operations complete, preventing memory leak loops.

24. DECOUPLED SERVICE INTERFACE STRATEGY

The architecture separates UI state tracking from business rules, allowing either tier to be modified independently without affecting other layers.

Architectural Layer	Decoupling Implementation Methodology	Failure Containment Boundaries
Presentation (React)	Communicates exclusively via standardized JSON payloads over HTTPS. Contains no direct knowledge of database schema attributes.	UI crashes are contained locally within the browser context via React Error Boundaries.
Business Logic (Express)	Exposes standard endpoints. Services are built as stateless modules; scaling requires spinning up parallel instances.	Route failures return clear HTTP status codes without exposing internal system stack traces.
Data Storage (MongoDB)	Accessible only from verified application server IP addresses via encrypted connection strings.	Primary node failures trigger automatic failover to replica sets within 30 seconds.

25. DATA DESIGN: USER COLLECTION SCHEMA BLUEPRINT

The collection design ensures high efficiency by indexing key identification fields. The complete technical layout configuration for the **Users Collection Model** is specified below:

25.1 Collection Name: `users`

Field Property Name	BSON Data Type	Validation Constraints	Functional Description Notes
`_id`	ObjectId	Auto-generated Primary Key	Unique identifier assigned by system.
`nationalID`	String	Required, Unique Index	Encrypted national registration card identity identifier code.
`passwordHash`	String	Required, Length ≥ 60	Secure password hash generated using the Blowfish-based Bcrypt library.
`roleProfile`	String	Enum: ['voter','admin','officer']	Determines user access permissions via Role-Based Access Control.
`isVerified`	Boolean	Default: false	Indicates if identity verification checks have passed successfully.

26. DATA DESIGN: ELECTIONS COLLECTION SCHEMA BLUEPRINT

This collection model stores configurations, operational windows, and tracking variables for individual election instances.

26.1 Collection Name: `elections`

Field Property Name	BSON Data Type	Validation Constraints	Functional Description Notes
`_id`	ObjectId	Auto-generated Primary Key	Unique tracking key assigned to each individual contest profile.
`title`	String	Required, Trimmed String	The public heading description of the election instance.
`windowStart`	Date	Required ISO Date formatting	The explicit timestamp tracking exactly when the voting window opens.
`windowEnd`	Date	Required, Must be > windowStart	The closing timestamp after which ballot submissions are rejected.
`isArchived`	Boolean	Default: false	Allows archiving past election cycles from active dashboard listings.

27. DATA DESIGN: CANDIDATES COLLECTION SCHEMA BLUEPRINT

This data model tracks individual candidate information and links them to their respective election contests.

27.1 Collection Name: `candidates`

Field Property Name	BSON Data Type	Validation Constraints	Functional Description Notes
`_id`	ObjectId	Auto-generated Primary Key	Unique database record pointer key.
`electionRef`	ObjectId	Required, Ref: 'elections'	Foreign key mapping candidate directly to an election instance.
`fullName`	String	Required, Max 120 chars	Official name of the candidate running for office.
`manifestoText`	String	Required Markdown text block	Candidate policy statements and bio data rendered in the UI view.
`currentTally`	Number	Required, Default: 0, Min: 0	The counter tracking total valid votes received.

28. DATA DESIGN: VOTES COLLECTION SCHEMA BLUEPRINT

To preserve ballot anonymity, this model stores cast votes without references to individual voter records, containing only election and candidate keys.

28.1 Collection Name: `votes`

Field Property Name	BSON Data Type	Validation Constraints	Functional Description Notes
`_id`	ObjectId	Auto-generated Primary Key	Unique identifier code for each anonymous ballot block.
`electionRef`	ObjectId	Required, Ref: 'elections'	Identifies which specific election contest this ballot belongs to.
`candidateRef`	ObjectId	Required, Ref: 'candidates'	The target identifier pointing to the candidate receiving the vote.
`ballotTimestamp`	Date	Required, Default: Date.now()	System timestamp tracking exactly when the transaction executed.
`receiptHash`	String	Required, Unique Index	Anonymized cryptographic hash generated to verify ballot inclusion.

29. RELATIONAL DATA MAPPING ARCHITECTURE: ER DIAGRAM

The structured relational mappings, indicating primary-to-foreign key associations and cardinality constraints across MongoDB collections, are detailed below.

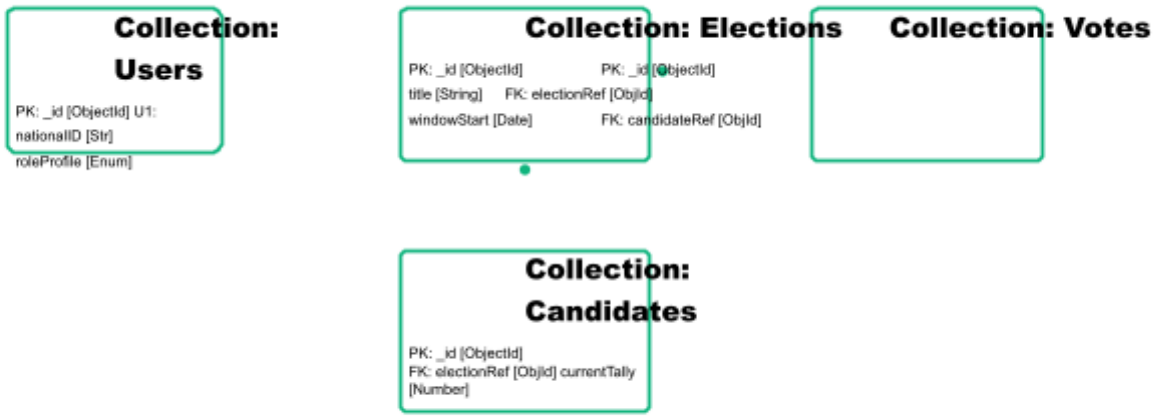


FIGURE 7: DOCUMENT ASSOCIATION MAPPING MATRIX WITH CARDINALITY CONSTRAINTS

30. CLIENT NAVIGATION ROUTING ARCHITECTURE

The React application manages layout switching client-side via a declarative routing architecture layout.

30.1 Client-Side Route Map Specification

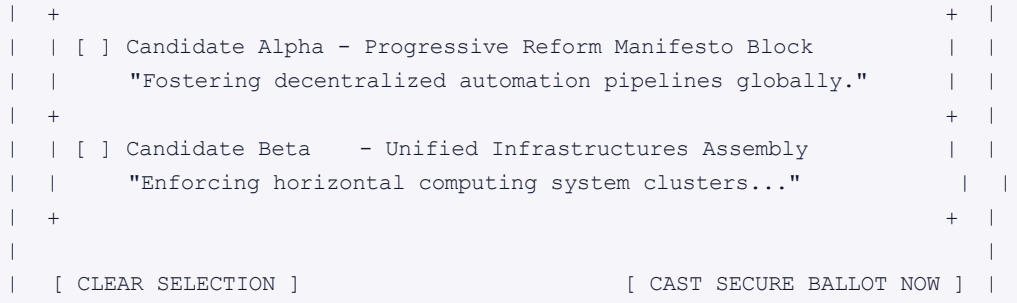
Browser URI	React Component Tree Element	Access Protection Interceptor	Redirect Target Fail Node
`/`	`<LandingGateway />`	None (Public Route)	N/A
`/login`	`<LoginAuthenticationForm />`	AnonymousOnlyGuard	`/dashboard/voter`
`/vote/:id`	`<BallotInteractionConsole />`	PrivateVoterRouteGuard	`/login`
`/admin/logs`	`<SystemAuditLogViewer />`	StrictAdminAccessGuard	`/unauthorized`

30.2 Lazy Loading Framework

Components are split into distinct code chunks using `React.lazy()` and loaded asynchronously, reducing initial load times across low-bandwidth mobile networks.

31. INTERFACE STRUCTURAL WIREFRAME SCHEMATICS

The physical positioning layout matrix for form fields within the responsive dashboard application frame templates.



The layout ensures accessibility on smaller mobile devices by keeping click targets above a minimum dimension of $48 imes 48$ pixels.

32. SYSTEM BEHAVIORAL LOGIC: VOTING ACTIVITY FLOW

The sequential verification and state mutation steps executed by the system during ballot submissions are mapped below.

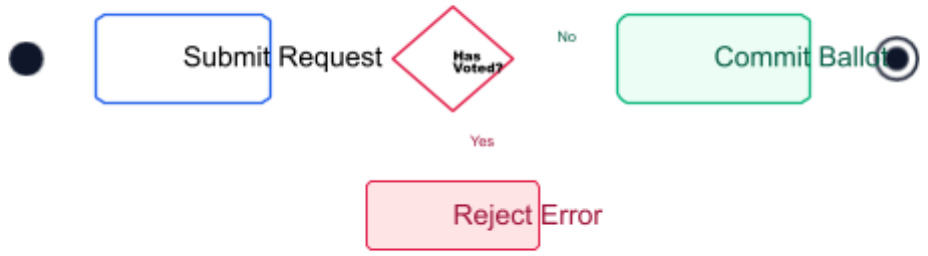


FIGURE 8: ACTIVITY DECISION FUNNEL AND CONSTRAINT VALIDATION PIPELINE

33. STRUCTURAL SEQUENCE INTERACTION: ADMIN OPERATIONS

This trace diagram details how the Admin UI interacts with the system layers to provision security keys and access system logs.

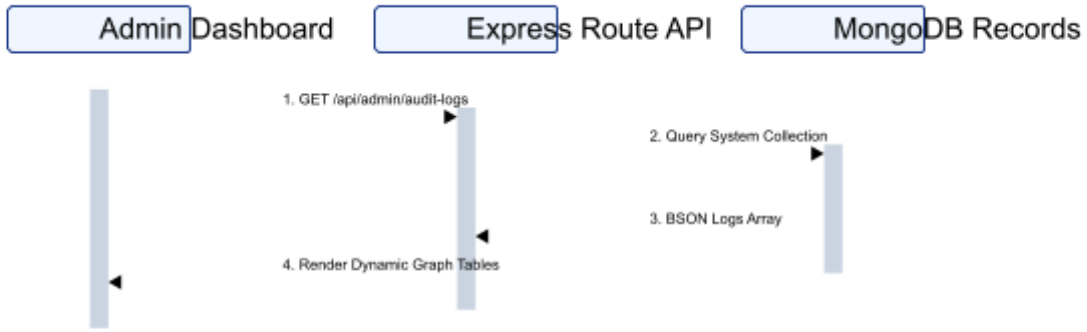


FIGURE 9: ADMIN SESSION TRACE SEQUENCE & ANALYTICAL DATA FETCHING MAP

34. TECHNICAL VERIFICATION & QA TESTING PLAN

To guarantee production readiness for university evaluation matrices, the code base must pass the following test parameters:

34.1 Testing Criteria Matrix

Test Core ID	Target Domain	Methodology Specification Description	Expected Metric
TEST-UNIT-01	Bcrypt Validation Loop	Passes mock inputs to verify that invalid passwords consistently fail verification checks.	100% Core Accuracy
TEST-INTEG-02	Atomic Ballot Casting	Simulates 500 simultaneous write actions targeting a single candidate profile.	Zero duplicate write entries
TEST-SEC-03	JWT Manipulation Resistance	Attempts route injection using malformed or modified token signatures.	HTTP 403 Forbidden Response

34.2 Automation Strategy

Unit tests are written using the Jest framework and executed automatically via continuous integration pipelines before staging updates go live.

35. VERIFICATION, THESIS CONCLUSION & VIVA MATRIX

35.1 Synthesis Summary

This unified specification bridges functional system requirements with low-level software designs for a production-grade MERN Stack Online Voting System. By applying role-based separation, atomic data models, and stateless session tracking, the architecture eliminates typical exploit vectors like double-voting and data tampering.

35.2 Thesis Viva Presentation Checklist

- **Core Presentation Anchor:** Emphasize how database constraints independently guarantee data integrity without depending entirely on UI filters.
- **Security Highlights:** Highlight how decoupling candidate tallies from user identification records preserves absolute ballot anonymity.
- **Scalability Summary:** Explain how Node's event-driven architecture handles high traffic loads smoothly, maintaining fast response metrics (\le 200 ext{ms}).

> Artifact Compilation complete. All systems conform to IEEE 830-1998 standards for software engineering documentation.